Ronak Katariya
D20A / 29

Experiment 9

AIM: Implement the Private Ethereum Blockchain using Geth

THEORY:

Q1: What is Geth?

Geth (Go-Ethereum) is one of the most widely used Ethereum clients, written in the Go programming language. It allows users to run an Ethereum node, mine Ether, and interact with the blockchain through JSON-RPC, command-line, or JavaScript consoles.

Q2: Significance of a Private Ethereum Network

Setting up a Private Ethereum Network involves creating a customized blockchain environment isolated from the public Ethereum network. It enables developers to test smart contracts, consensus mechanisms, and network behavior without incurring gas costs or depending on public validators.

Q3: Steps for creating a Private Ethereum Network

Steps Involved:

- **Choosing a Network ID:**
  Each private blockchain must have a unique network ID to differentiate it from public or other private networks.
- **Choosing a Consensus Algorithm:**
  Ethereum supports different consensus mechanisms such as Proof of Work (PoW) and Proof of Authority (PoA). In private networks, Clique (PoA) is often used for faster block confirmation.
- **Creating a Genesis Block:**
  The genesis block is the first block of the blockchain, defining network parameters such as difficulty, gas limit, initial account balances, and consensus settings.
- **Initializing the Geth Database:**
  The Geth init command initializes the blockchain using the genesis.json file, preparing the data directory for node operation.
- **Setting up Networking:**
  Nodes communicate using enodes (Ethereum Node URLs) that contain IP addresses and public keys for peer discovery and connection.
- **Running the Member Nodes:**
  Each participant runs a Geth node that connects to the private network and participates in block validation and transaction processing.
- **Running a Signer (in Clique):**
  In the Clique consensus, designated signers validate and sign blocks. These accounts are pre-configured in the genesis file to ensure trusted block production.

NOTE: Download the genesis file and edit the account details, including the public keys of all participating peers in the network.

PROCEDURE

Step 1: Install Geth

wget https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.13.14-2bd6bd01.tar.gz tar -xvf geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
mkdir -p ~/bin

cp geth-linux-amd64-1.13.14-2bd6bd01/geth ~/bin/geth113
export PATH="$HOME/bin:$PATH"
geth113 version

```
PS C:\Users\2022n> geth version
Geth
Version: 1.13.14-stable
Git Commit: 2bd6bd01d2e8561dd7fc21b631f4a34ac16627a1
Git Commit Date: 20240227
Architecture: amd64
Go Version: go1.21.6
Operating System: windows
GOPATH=
GOROOT=
PS C:\Users\2022n>
```

Step 2: Create folders and accounts for both nodes

```
PS C:\Users\2022n\private-eth> geth account new --datadir node3
INFO [04-11|19:09:08.105] Maximum peer count                       ETH=50 total=50
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key:   0x44BD5AAd8E5baC3689bF86AC2902e71aC1c02Dc9
Path of the secret key file: node3\keystore\UTC--2026-04-11T13-39-12.606917700Z--44bd5aad8e5bac3689bf86ac2902e71ac1c02dc9

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!

PS C:\Users\2022n\private-eth> geth account new --datadir node4
INFO [04-11|19:09:17.550] Maximum peer count                       ETH=50 total=50
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key:   0x78d8C562008056B60AEA9572b655FB7AC8994bad
Path of the secret key file: node4\keystore\UTC--2026-04-11T13-39-20.744427300Z--78d8c562008056b60aea9572b655fb7ac8994bad

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!

PS C:\Users\2022n\private-eth>
```

Step 3: Create password files

Step 4: Create genesis.json and verify it looks correct {

"config": {

"chainId": 1234,

"homesteadBlock": 0,

"eip150Block": 0,

"eip155Block": 0,

"eip158Block": 0,

"byzantiumBlock": 0,
"constantinopleBlock": 0,
"petersburgBlock": 0,
"clique": {
"period": 5,
"epoch": 30000
}
},
"difficulty": "1",
"gasLimit": "8000000",
"extraData":
"0x0000000000000000000000000000000000000000000000000000000000000000445c4d7
d4257ca2299761fbcbef2dc2bbf029b350000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000",
"alloc": {
"445c4d7d4257ca2299761fbcbef2dc2bbf029b35": {
"balance": "10000000000000000000"
},
"0b8c32888a516fb55d634a3bfba75843b9d258c4": {
"balance": "10000000000000000000"
}
}
}

Step 5: Initialize both nodes

```
PS C:\Users\2022n\private-eth> geth --datadir node1 init genesis.json
INFO [04-11|19:13:40.385] Maximum peer count                       ETH=50 total=50
INFO [04-11|19:13:40.441] Set global gas cap                       cap=50,000,000
INFO [04-11|19:13:40.441] Initializing the KZG library             backend=gokzg
INFO [04-11|19:13:40.470] Using pebble as the backing database
INFO [04-11|19:13:40.470] Allocated cache and file handles         database=C:\Users\2022n\private-eth\node1\geth\chaindata cache=16.00MiB handles=16
INFO [04-11|19:13:40.526] Opened ancient database                  database=C:\Users\2022n\private-eth\node1\geth\chaindata\ancient/chain readonly=false
INFO [04-11|19:13:40.526] State scheme set to already existing     scheme=hash
INFO [04-11|19:13:40.527] Successfully wrote genesis state         database=chaindata hash=e13161..caa542
INFO [04-11|19:13:40.529] Using pebble as the backing database
INFO [04-11|19:13:40.529] Allocated cache and file handles         database=C:\Users\2022n\private-eth\node1\geth\lightchaindata cache=16.00MiB handles=16
INFO [04-11|19:13:40.623] Opened ancient database                  database=C:\Users\2022n\private-eth\node1\geth\lightchaindata\ancient/chain readonly=Fals
e
INFO [04-11|19:13:40.624] State scheme set to already existing     scheme=hash
INFO [04-11|19:13:40.624] Successfully wrote genesis state         database=lightchaindata hash=e13161..caa542
```

Step 6: Run Node 1 (signer)

```
PS C:\Users\2022n> cd .\private-eth\
PS C:\Users\2022n\private-eth> geth --datadir node1 --networkid 1234 --port 30303 --nodiscover --unlock 0x665C4E7D4257cA2209761FbCBeF3dC2bbf029b25 --passwor
d node1\password.txt --allow-insecure-unlock console
INFO [04-11|18:46:48.852] Maximum peer count                       ETH=50 total=50
INFO [04-11|18:46:48.857] Set global gas cap                       cap=50,000,000
INFO [04-11|18:46:48.857] Initializing the KZG library             backend=gokzg
INFO [04-11|18:46:48.889] Allocated trie memory caches             clean=154.00MiB dirty=256.00MiB
INFO [04-11|18:46:48.889] Using pebble as the backing database
INFO [04-11|18:46:48.889] Allocated cache and file handles         database=C:\Users\2022n\private-eth\node1\geth\chaindata cache=512.00MiB handles=8192
INFO [04-11|18:46:48.986] Opened ancient database                  database=C:\Users\2022n\private-eth\node1\geth\chaindata\ancient/chain readonly=false
INFO [04-11|18:46:48.910] State scheme set to already existing     scheme=hash
INFO [04-11|18:46:48.928] Initializing Ethereum protocol           network=1234 dbversion=8
INFO [04-11|18:46:48.928]
INFO [04-11|18:46:48.929] ----------------------------------------------------------------------------------------------------------------------
-----------------------
INFO [04-11|18:46:48.929] Chain ID:  1234 (unknown)
INFO [04-11|18:46:48.929] Consensus: Clique (proof-of-authority)
INFO [04-11|18:46:48.929]
INFO [04-11|18:46:48.929] Pre-Merge hard forks (block based):
INFO [04-11|18:46:48.929]  - Homestead:                    #0        (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrad
es/homestead.md)
INFO [04-11|18:46:48.929]  - Tangerine Whistle (EIP 150): #0        (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrad
es/tangerine-whistle.md)
INFO [04-11|18:46:48.929]  - Spurious Dragon/1 (EIP 155): #0        (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrad
es/spurious-dragon.md)
INFO [04-11|18:46:48.929]  - Spurious Dragon/2 (EIP 158): #0        (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrad
es/spurious-dragon.md)
INFO [04-11|18:46:48.929]  - Byzantium:                    #0        (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrad
es/byzantium.md)
INFO [04-11|18:46:48.929]  - Constantinople:               #0        (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrad
es/constantinople.md)
INFO [04-11|18:46:48.930]  - Petersburg:                   #0        (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrad
es/petersburg.md)
INFO [04-11|18:46:48.930]  - Istanbul:                     #<nil> (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/
```

Step 7: Open new terminal tab and run Node 2 (member)

```
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\2022n> cd .\private-eth\
PS C:\Users\2022n\private-eth> geth --datadir node2 --networkid 1234 --port 30306 --nodiscover --ipcpath \\.\pipe\geth2.ipc --authrpc.port 8552 console
INFO [04-11|18:47:00.045] Maximum peer count                       ETH=50 total=50
INFO [04-11|18:47:00.048] Set global gas cap                       cap=50,000,000
INFO [04-11|18:47:00.048] Initializing the KZG library             backend=gokzg
INFO [04-11|18:47:00.089] Allocated trie memory caches             clean=154.00MiB dirty=256.00MiB
INFO [04-11|18:47:00.090] Using pebble as the backing database
INFO [04-11|18:47:00.090] Allocated cache and file handles         database=C:\Users\2022n\private-eth\node2\geth\chaindata cache=512.00MiB handles=8192
INFO [04-11|18:47:00.904] Opened ancient database                  database=C:\Users\2022n\private-eth\node2\geth\chaindata\ancient/chain readonly=false
INFO [04-11|18:47:00.911] State scheme set to already existing     scheme=hash
INFO [04-11|18:47:00.917] Initializing Ethereum protocol           network=1234 dbversion=8
INFO [04-11|18:47:00.917]
INFO [04-11|18:47:00.917] ----------------------------------------------------------------------------------------------------------------------
-----------------------
INFO [04-11|18:47:00.917] Chain ID:  1234 (unknown)
INFO [04-11|18:47:00.917] Consensus: Clique (proof-of-authority)
INFO [04-11|18:47:00.918]
INFO [04-11|18:47:00.918] Pre-Merge hard forks (block based):
INFO [04-11|18:47:00.918]  - Homestead:                    #0        (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrad
es/homestead.md)
INFO [04-11|18:47:00.918]  - Tangerine Whistle (EIP 150): #0        (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrad
es/tangerine-whistle.md)
INFO [04-11|18:47:00.918]  - Spurious Dragon/1 (EIP 155): #0        (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrad
es/spurious-dragon.md)
INFO [04-11|18:47:00.918]  - Spurious Dragon/2 (EIP 158): #0        (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrad
es/spurious-dragon.md)
INFO [04-11|18:47:00.918]  - Byzantium:                    #0        (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrad
es/byzantium.md)
INFO [04-11|18:47:00.918]  - Constantinople:               #0        (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrad
es/constantinople.md)
INFO [04-11|18:47:00.918]  - Petersburg:                   #0        (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrad
es/petersburg.md)
INFO [04-11|18:47:00.918]  - Istanbul:                     #<nil> (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/
istanbul.md)
INFO [04-11|18:47:00.918]  - Berlin:                       #<nil> (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/
berlin.md)
```

Step 8: Connect the two nodes

In Node 1 console, get its enode

```
> admin.nodeInfo.enode
"enode://[redacted]@143.138.199:30303"
```

Now, switch to Node 2 console and paste the Node 1 enode

```
> admin.addPeer("enode://[redacted]@143.138.199:30303"
)
true
```

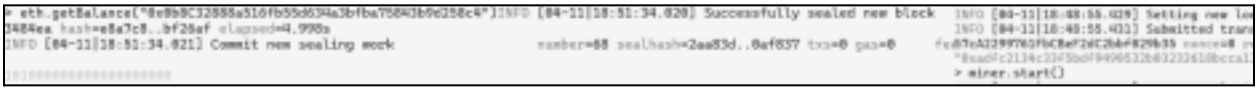
In Node 2 console, check they are connected

```
> admin.peers
[{
    caps: ["eth/68", "eth/69", "snap/1"],
    enode: "enode://[redacted]",
    id: "[redacted]",
    name: "Betherumud/v1.16.0+31c8f1b7/linux-amd64/dotnet10.0.1",
    network: {
      inbound: false,
      localAddress: "10.88.0.4:55566",
      remoteAddress: "47.187.149.99:31404",
      static: false,
      trusted: false
    },
    protocols: {
      eth: "handshake",
      snap: "handshake"
    }
}]
```

Step 9: Send a transaction

In Node 1 console, send transaction to Node 2

and check the balance of Node 2



CONCLUSION:

Through this experiment, two smart contracts - OrderRecord and ReviewVerification, were successfully created and deployed on the Ethereum Sepolia test network using Solidity and Remix IDE. The experiment demonstrated how blockchain technology can be integrated into a full-stack web application to permanently and transparently record order transactions and product reviews. The use of the Sepolia test network and MetaMask allowed safe testing without real funds. Overall, this experiment provided practical understanding of designing, compiling, deploying, and testing smart contracts on the Ethereum blockchain.